

1. Project Title

SMARTCAMPUS ERP — AI-Powered Student Performance Analytics, Academic Management, Coding, Placement and Career Management System

Build a complete, production-structured college ERP web application from scratch.

The system must combine traditional college ERP functionality with:

- AI-powered study assistance
- Student performance analytics
- Coding practice and code execution
- Coding contests and leaderboard
- Placement management
- Resume builder
- Interview preparation
- Interview scheduling
- Real-time notifications
- Role-based dashboards
- Academic monitoring

The application must be fully functional end-to-end, not a UI prototype. Every frontend feature that requires backend functionality must have a corresponding backend API, database model, validation, error handling and persistence.

2. Primary Objective

The system should provide a single platform for:

1. Student academic management
2. Faculty academic management
3. College administration
4. Attendance management
5. Marks and examination management
6. Student performance analytics
7. AI-based study assistance
8. Coding practice and evaluation
9. Coding competitions

10. Placement management
11. Resume creation
12. Interview preparation
13. Interview scheduling
14. Notifications and announcements
15. Real-time updates

The system should be designed so that a college can manage its students, faculty, courses, academic performance and career activities from one centralized platform.

3. Mandatory Technology Stack

Frontend

Use:

- **React.js**
- **Vite**
- JavaScript or TypeScript
- React Router
- Axios
- Chart.js / react-chartjs-2
- CSS or a professional component/UI library
- WebSocket client for real-time notifications

Recommended frontend structure:

```
frontend/
├── src/
│   ├── components/
│   ├── pages/
│   ├── layouts/
│   ├── services/
│   ├── hooks/
│   ├── context/
│   ├── utils/
│   ├── routes/
│   ├── assets/
│   └── App.jsx
├── package.json
└── vite.config.js
```

The frontend must be responsive and work on:

- Desktop
- Laptop
- Tablet
- Mobile

4. Backend

Use:

- **Java 21**
- **Spring Boot 4.1.x**
- Maven
- Spring Web
- Spring Data JPA
- Hibernate
- Spring Security
- Bean Validation
- Spring WebSocket
- Lombok where useful

Backend architecture must follow:

4. Backend

Use:

- **Java 21**
- **Spring Boot 4.1.x**
- Maven
- Spring Web
- Spring Data JPA
- Hibernate
- Spring Security
- Bean Validation
- Spring WebSocket
- Lombok where useful

Backend architecture must follow:

Controller



Service

↓
Repository
↓
Entity
↓
MySQL

Do not place business logic directly inside controllers.

Recommended structure:

```
backend/  
├── src/  
│   ├── main/  
│   │   ├── java/  
│   │   │   ├── smartcampus/  
│   │   │   │   ├── config/  
│   │   │   │   ├── controller/  
│   │   │   │   ├── dto/  
│   │   │   │   ├── entity/  
│   │   │   │   ├── exception/  
│   │   │   │   ├── repository/  
│   │   │   │   ├── security/  
│   │   │   │   ├── service/  
│   │   │   │   └── util/  
│   │   └── resources/  
│   │       ├── application.properties  
│   │       └── ...
```

5. Database

Use:

MySQL 8.0+

Database name:

Smartcampus

Use JPA/Hibernate for ORM.

Database design must use:

- Primary keys
- Foreign keys
- Unique constraints
- NOT NULL constraints where appropriate
- Indexes on frequently searched fields
- Proper relationships
- Timestamps
- Enumerations where appropriate

Do not store passwords in plaintext.

Use BCrypt password hashing.

6. Authentication and Authorization

Implement complete authentication using:

Login

Users authenticate using:

Email

Password

Supported roles:

STUDENT

FACULTY

ADMIN

Authentication flow:

User

↓

Login

↓

Verify email

↓

Verify BCrypt password

↓

Generate JWT

↓

Return JWT + user information + role

Use JWT for stateless authentication.

JWT should contain at minimum:
email/user identifier
role
issued time
expiration time

Use:

- JJWT
- JWT secret stored through environment variables in production
- Configurable expiration time

JWT request format:

Authorization: Bearer <JWT>

7. JWT Filter

Create a JWT authentication filter using Spring Security.

For every protected request:

HTTP Request
↓
Authorization Header
↓
Extract Bearer Token
↓
Validate JWT
↓
Extract User
↓
Load User
↓
Determine Role
↓
Create Authentication
↓
SecurityContext
↓
Controller

Invalid or expired tokens must result in appropriate authentication errors.

Do not silently authenticate invalid tokens.

8. Role-Based Authorization

Implement authorization at backend level.

STUDENT

Can access:

- Own profile
- Own attendance
- Own marks
- Own academic analytics
- AI Study Assistant
- Coding playground
- Coding contests
- Leaderboard
- Placement portal
- Resume builder
- Interview preparation
- Own interview schedules
- Own notifications

A student must never be able to access another student's private academic information by modifying an ID in the URL.

FACULTY

Can access:

- Faculty profile
- Assigned courses
- Assigned subjects
- Students belonging to their courses/subjects
- Attendance management
- Marks management
- Academic monitoring
- Student performance analytics
- Announcements/notifications

Faculty should only be allowed to modify academic information for courses/subjects they are authorized to manage.

ADMIN

Can access:

All students

All faculty

Departments

Courses

Subjects

Academic configuration

Attendance

Marks

Placement management

Users

Announcements

System analytics

Coding contests

Interview schedules

System-level notifications

Admin has system-level privileges.

9. Authentication APIs

Implement APIs similar to:

POST /api/auth/register
POST /api/auth/login
POST /api/auth/forgot-password
POST /api/auth/verify-otp
POST /api/auth/reset-password
GET /api/auth/me

Registration must validate:

- Name
- Email
- Password
- Phone
- Role

Prevent duplicate emails.

Passwords must be BCrypt encrypted before storage.

10. OTP Password Reset

Implement email-based password reset.

Workflow:

User clicks Forgot Password



Enter email



Backend verifies account



Generate secure OTP



Send OTP to email



User enters OTP



Verify OTP



Allow new password



BCrypt hash new password



Update database

OTP must:

- Expire after configurable time
- Be single-use
- Have limited verification attempts
- Not be stored as plaintext if possible
- Not expose whether an email exists in a way that enables account enumeration

Use SMTP/email service through environment variables.

11. Core User Entity

Create a central **User** entity.

Fields:

id
name
email
password
role
phone
createdAt
updatedAt

Email must be unique.

Role must be:

STUDENT
FACULTY
ADMIN

Do not return the password field in API responses.

Use DTOs for API responses.

12. Student Management

Create a Student profile linked to User.

Student fields should include:

id
userId
registerNumber
departmentId
courseId
year
semester
section
dateOfBirth
gender
address
profileImage
admissionYear
graduationYear
createdAt
updatedAt

Register number must be unique.

Students should have a profile page where they can view/edit permitted personal information.

13. Faculty Management

Faculty entity:

id
userId
employeeId
departmentId
designation
joiningDate
qualification
createdAt
updatedAt
Employee ID must be unique.

14. Department Management

Department entity:

id

name

code

description

createdAt

updatedAt

Department code must be unique.

Examples:

CSE

ECE

EEE

MECH

CIVIL

IT

The system must allow an administrator to create, edit and deactivate departments.

15. Course Management

Course entity:

id

name

code

departmentId

duration

description

active

createdAt

updatedAt

Examples:

B.Tech Computer Science

B.E. Electronics

B.Tech Information Technology

16. Subject Management

Subject entity:

id

name

code

credits

semester

courseId

departmentId

description

active

Course-to-subject relationship:

Course 1 — * Subject

17. Enrollment

Students and subjects have a many-to-many relationship.

Do not directly create an uncontrolled many-to-many relationship.

Use an Enrollment entity:

id

studentId

subjectId

academicYear

semester

enrollmentDate

status

Relationship:

Student * — * Subject

Enrollment

18. Attendance Module

Faculty should be able to record attendance for authorized subjects.

Attendance entity:

id

studentId

subjectId

facultyId

date

status

remarks

createdAt

Status:

PRESENT

ABSENT

LATE

Features:

- Mark attendance
- Update attendance with permission
- View attendance history
- Filter by subject
- Filter by date
- Calculate attendance percentage
- Student attendance dashboard
- Faculty attendance dashboard
- Low-attendance warning
- Attendance reports

Calculation:

Attendance Percentage =

$(\text{Present Classes} / \text{Total Classes}) \times 100$

The system must correctly handle:

- No attendance records
- Cancelled classes
- Multiple subjects
- Different semesters
- Different academic years

19. Marks / Examination Module

Marks entity:

id

studentId

subjectId

facultyId

examType

marksObtained

maximumMarks

examDate

remarks

createdAt

updatedAt

Exam types:

INTERNAL

MIDTERM

ASSIGNMENT

QUIZ

FINAL

PRACTICAL

Features:

- Enter marks
- Update marks
- View marks
- Subject-wise marks
- Exam-wise marks
- Semester performance
- Percentage calculation
- Grade calculation
- GPA/CGPA calculation
- Performance trends

Validate:

marksObtained <= maximumMarks

marksObtained >= 0

20. Grade Calculation

Provide configurable grading rules.

Example:

90–100 → A+

80–89 → A

70–79 → B+

60–69 → B

50–59 → C

40–49 → D

<40 → F

Do not hard-code these permanently. Allow admin configuration if possible.

21. Student Academic Dashboard

Student dashboard should display:

Student profile

Attendance percentage

Subject-wise attendance

Current semester marks

Overall percentage

GPA/CGPA

Recent exams

Recent assignments

Academic performance trend

Low attendance warnings

Upcoming academic events

Notifications

Charts:

- Attendance chart
- Subject performance chart
- Semester performance line graph
- Marks distribution
- GPA trend

Use Chart.js.

22. Faculty Dashboard

Faculty dashboard should show:

Assigned courses

Assigned subjects

Total students

Attendance summary

Pending marks

Recent academic activity

Student performance statistics

Low attendance students

Upcoming classes/exams

Notifications

Faculty should have filters:

Course

Subject

Semester

Section

Academic year

23. Admin Dashboard

Admin dashboard should provide system-wide analytics:

Total students

Total faculty

Total departments

Total courses

Total subjects

Average attendance

Average academic performance

Students below attendance threshold

Top-performing students

Students at academic risk

Placement statistics

Coding statistics

Charts:

- Students by department
- Attendance by department
- Performance by department
- Placement statistics
- Contest participation
- Coding performance

24. Academic Performance Analytics

Create a dedicated analytics service.

It should calculate:

Attendance percentage

Average marks

Subject performance

Semester performance

GPA

CGPA

Performance trend

Academic risk

Student performance classification:

EXCELLENT

GOOD

AVERAGE

AT_RISK

Risk detection should be based on configurable thresholds.

Example:

Low attendance

+

Low marks

=

Academic Risk

The exact thresholds should be configurable rather than hard-coded.

25. AI Study Assistant

Integrate an AI API.

Preferred architecture:

React

↓

Spring Boot

↓

AI Service

↓

Gemini/OpenAI API

Do **not** expose the AI API key in React.

The backend must call the AI provider.

Features:

Student can ask:

- Subject questions
- Concept explanations
- Programming questions
- Study planning questions
- Exam preparation questions
- Summarization requests
- Practice question generation
- Quiz generation

The assistant should support contextual prompts such as:

Subject

Semester

Student performance

Weak subjects

Upcoming exam

The AI assistant should be able to generate:

Study plan

Topic explanation

Practice questions

MCQs

Revision schedule

Programming explanations

26. AI Study Plan

Based on academic data:

Marks

Attendance

Weak subjects

Upcoming exams

Available study time

generate a personalized study plan.

Example:

Monday

DBMS – 1 hour

Data Structures – 1.5 hours

Tuesday

Operating Systems – 1 hour

Programming – 1 hour

AI recommendations should be advisory and editable by the student.

27. AI Conversation History

Store AI conversations.

Entities:

AIConversation

AIMessage

AIConversation:

id

userId

title

createdAt

updatedAt

AIMessage:

id

conversationId

sender

message

timestamp

Sender:

USER

AI

Students should be able to:

- Create conversation
- Continue conversation
- Rename conversation
- Delete conversation
- View history

Do not store sensitive API credentials in the database.

28. Coding Playground

Implement a browser-based coding environment.

Supported languages initially:

Java

C++

The system should use **Judge0** for remote code execution.

Architecture:

React Code Editor



Spring Boot



Judge0 API



Code Execution



Result



Spring Boot



React

The backend must never directly execute arbitrary student code on the application server.

29. Coding Submission

Submission entity:

id

studentId

language

sourceCode

stdin

stdout

stderr

status

executionTime

memory

createdAt

Statuses:

PENDING

PROCESSING

ACCEPTED

WRONG_ANSWER

COMPILATION_ERROR

RUNTIME_ERROR

TIME_LIMIT_EXCEEDED

MEMORY_LIMIT_EXCEEDED

Students should be able to view submission history.

30. Coding Problems

Problem entity:

id

title

description

difficulty

constraints

inputFormat

outputFormat

sampleInput

sampleOutput

createdBy

active

createdAt

updatedAt

Difficulty:

EASY

MEDIUM

HARD

Admin/faculty-authorized users can create problems.

Students can:

- View problems
- Filter by difficulty
- Submit solutions
- View results
- Track solved problems

31. Coding Contests

Contest entity:

id

title

description

startTime

endTime

duration

status

createdBy

createdAt

Contest statuses:

UPCOMING

LIVE

COMPLETED

Contest participation:

ContestParticipant

Fields:

id

contestId

studentId

joinedAt

score

rank

Contest problem mapping:

ContestProblem

Leaderboard should calculate:

Score

Solved problems

Penalty/time

Rank

32. Coding Leaderboard

Leaderboard should support:

- Global leaderboard
- Contest leaderboard
- Student rank
- Problems solved
- Total score
- Recent submissions

Use WebSockets for live leaderboard updates where appropriate.

33. Placement Portal

Placement module should contain:

Company

id

name

logo

description

website

industry

Location

Job / Placement Drive :

id

companyId

title

description

jobType

location

salary

eligibilityCriteria

minimumCGPA

minimumPercentage

allowedDepartments

applicationDeadline

driveDate

status

createdAt

Statuses:

OPEN

CLOSED

CANCELLED

34. Placement Eligibility

The system must automatically determine eligibility using:

CGPA

Percentage

Department

Graduation year

Backlogs if configured

Other company criteria

Student should see:

Eligible

Not Eligible

with a clear reason when not eligible.

35. Placement Application

Application entity:

id

studentId

jobId

resumeld

status

appliedAt

updatedAt

Statuses:

APPLIED

SHORTLISTED

ASSESSMENT

INTERVIEW

SELECTED

REJECTED

WITHDRAWN

Students can apply only if eligible and before the deadline.

Prevent duplicate applications.

36. Placement Admin

Admin can:

- Create companies
 - Create placement drives
 - Define eligibility
 - View applicants
 - Shortlist students
 - Update application status
 - Schedule interviews
 - View placement analytics
-

37. Resume Builder

Students should be able to create resumes from structured data.

Resume sections:

Personal Information

Career Objective

Education

Skills

Projects

Experience

Certifications

Achievements

Languages

Interests

Entities:

Resume

Education

Project

Experience

Certification

Skill

Achievement

Students should be able to:

- Create resume
- Edit resume
- Save multiple versions
- Preview resume
- Select template
- Export/download as PDF

The resume builder should use student profile data automatically where possible.

38. Interview Preparation

Create interview preparation module.

Categories:

Technical

HR

Behavioral

Coding

Aptitude

Company-specific

Features:

- Question bank
 - Practice questions
 - Answers/explanations
 - Mark question as completed
 - Bookmark questions
 - Track preparation progress
 - AI-generated practice questions
-

39. Interview Scheduling

Interview entity:

id

studentId

companyId

jobId

interviewer

scheduledDate

startTime

endTime

meetingLink

status

notes

createdAt

updatedAt

Statuses:

SCHEDULED

RESCHEDULED

COMPLETED

CANCELLED

Prevent conflicting interview schedules.

Students should see upcoming interviews in their dashboard.

40. Notifications

Create notification system.

Entity:

Notification

Fields:

id

userId

title

message

type

isRead

createdAt

Notification types:

ACADEMIC

ATTENDANCE

MARKS

PLACEMENT

INTERVIEW

CONTEST

SYSTEM

ANNOUNCEMENT

AI

Features:

- Notification center
 - Mark as read
 - Mark all as read
 - Delete notification
 - Unread count
-

41. Real-Time Notifications

Use:

Spring WebSocket

Architecture:

Backend Event



WebSocket



Connected React Client



Instant notification

Use real-time notifications for:

- Placement updates
- Interview schedule changes
- Contest updates
- Leaderboard updates
- Admin announcements
- Attendance warnings
- Important academic notifications

WebSocket authentication must respect the user's JWT identity.

42. Announcement System

Admin/faculty-authorized users can create announcements.

Announcement:

id

title

content

createdBy

targetRole

targetDepartment

priority

createdAt

expiresAt

Target options:

ALL

STUDENTS

FACULTY

DEPARTMENT

Announcements should appear on relevant dashboards and notifications.

43. Search and Filtering

Implement server-side search/filtering where datasets can become large.

Examples:

Student:

name

register number

department

year

semester

Faculty:

name

employee ID

department

Courses:

name

code

department

Placement:

company

job title

department

eligibility

status

Coding problems:

title

difficulty

language

44. Pagination

Large lists must use pagination.

Examples:

Students

Faculty

Marks

Attendance

Notifications

Jobs

Applications

Submissions

Coding Problems

Announcements

Backend should return pagination metadata:

content

page

size

totalElements

totalPages

45. API Design

Use RESTful APIs.

Examples:

/api/auth

/api/users

/api/students

/api/faculty

/api/admin

/api/departments

/api/courses

/api/subjects

/api/enrollments

/api/attendance

/api/marks

/api/analytics

/api/ai

/api/coding

/api/problems

/api/contests

/api/leaderboard

/api/companies

/api/jobs

/api/applications

/api/resumes

/api/interviews

/api/notifications

/api/announcements

Use appropriate HTTP methods:

GET retrieve

POST create

PUT update

PATCH partial update where appropriate

DELETE delete

Use meaningful HTTP status codes:

200 OK

201 CREATED

204 NO_CONTENT

400 BAD_REQUEST

401 UNAUTHORIZED

403 FORBIDDEN

404 NOT_FOUND

409 CONFLICT

422 UNPROCESSABLE_ENTITY

500 INTERNAL_SERVER_ERROR

46. DTO Architecture

Do not expose JPA entities directly from APIs.

Use DTOs:

Request DTO

Response DTO

Example:

RegisterRequest

LoginRequest

AuthResponse

StudentResponse

AttendanceRequest

AttendanceResponse

MarksRequest

MarksResponse

JobResponse

ApplicationResponse

Passwords must never appear in response DTOs.

47. Global Exception Handling

Create:

GlobalExceptionHandler

Handle:

- Validation errors
- Resource not found
- Duplicate email
- Duplicate register number
- Invalid role
- Invalid credentials
- Unauthorized access
- Forbidden access
- Database errors
- AI service failures
- Judge0 failures
- File upload errors

Return consistent JSON:

```
{  
  "timestamp": "...",  
  "status": 404,  
  "error": "NOT_FOUND",  
  "message": "Student not found",
```

```
"path": "/api/students/10"  
}
```

Do not expose stack traces to frontend users.

48. Validation

Backend validation is mandatory even if frontend validation exists.

Validate:

- Email format
- Required fields
- Password length
- Phone format
- Numeric ranges
- Marks range
- Date/time validity
- Duplicate records
- Eligibility
- File types
- File size

Never rely only on React validation.

49. File Management

Files may be required for:

- Profile images
- Resume exports
- Resume attachments
- Company logos
- Certifications

Do not store arbitrary large files directly in MySQL.

Use filesystem/object storage abstraction.

Store only metadata/path in database.

Validate:

File type

File size

File name

Prevent malicious file uploads.

50. Frontend Authentication

React must have:

Login page

Registration page

Forgot password

OTP verification

Reset password

Logout

Protected routes

Role-based routes

After login:

JWT

↓

Authentication state

↓

User role

↓

Dashboard

Axios should automatically attach:

Authorization: Bearer <JWT>

to protected API requests.

Handle expired JWT by redirecting to login.

51. Frontend Route Structure

Example:

/login

/register

/forgot-password

/reset-password

/student/dashboard

/student/profile

/student/attendance

/student/marks

/student/analytics

/student/ai-assistant

/student/coding

/student/contests

/student/leaderboard

/student/placements

/student/resume

/student/interviews

/student/notifications

/faculty/dashboard

/faculty/students

/faculty/courses

/faculty/attendance

/faculty/marks

/faculty/analytics

/admin/dashboard

/admin/students

/admin/faculty

/admin/departments

/admin/courses

/admin/subjects

/admin/placements

/admin/companies

/admin/contests

/admin/announcements

/admin/analytics

Unauthorized users must not be able to access another role's routes.

52. UI Requirements

Create a professional modern ERP interface.

Common layout:

Sidebar

Top Navigation

Main Content

Notifications

User Profile

Student dashboard should have cards for:

Attendance

CGPA

Subjects

Assignments

Placement

Coding

Notifications

Use responsive charts and tables.

Include:

- Loading states
- Empty states
- Error states
- Confirmation dialogs
- Toast notifications
- Form validation messages
- Pagination
- Search
- Filters

Do not create static dummy screens that aren't connected to APIs.

53. Data Integrity

The application must enforce business rules.

Examples:

Student

A student cannot:

- Modify another student's marks
- Modify another student's attendance
- View another student's private information
- Apply twice to the same job
- Submit after contest deadline

Faculty

Faculty cannot:

- Modify unrelated subjects
- Modify another faculty's course unless authorized
- Modify admin data

Admin

Admin can manage system-level data.

54. Academic Workflow

Complete workflow:

Admin creates Department



Admin creates Course



Admin creates Subjects



Faculty assigned to Subject



Students enrolled



Faculty records Attendance



Faculty records Marks



System calculates Performance



Student sees Dashboard



Analytics identifies weak areas



AI Study Assistant recommends study plan

55. Placement Workflow :

Admin creates Company



Admin creates Placement Drive



Eligibility criteria configured



System checks student eligibility



Eligible students see opportunity



Student applies



Resume selected



Application created



Admin shortlists



Assessment



Interview scheduled



Student notified



Interview completed



Application status updated



Placement analytics updated

56. Coding Workflow :

Student opens Coding Playground



Select language



Write code



Enter input



Submit



Spring Boot



Judge0



Compilation/execution



Result



Save submission



Display result

57. Contest Workflow :

Admin creates contest



Adds problems



Contest scheduled



Students register/join



Contest becomes LIVE



Students submit solutions



Judge0 evaluates



Scores calculated



Leaderboard updated



Contest ends



Final rankings stored

58. Interview Workflow :

Placement drive



Student shortlisted



Admin schedules interview



Student receives notification



Interview appears on dashboard



Interview conducted



Status updated



Student notified

59. AI Workflow :

Student



AI Study Assistant



Spring Boot AI Service



Build contextual prompt



Gemini/OpenAI API



AI response



Save conversation



Return response



Display in React

AI API credentials must only exist on the backend.

60. Analytics Workflow :

Attendance

Marks

Assignments

Coding

Placement



Analytics Service



Aggregations



Performance Metrics



Chart.js



Dashboard

Analytics should be calculated from real database data, not hard-coded numbers.

61. Security Requirements

Mandatory:

- BCrypt password hashing
- JWT authentication
- Role-based authorization
- Backend authorization
- Input validation
- SQL injection protection through JPA/parameterized queries
- CORS configuration
- Secure HTTP headers where appropriate
- API key protection
- Environment variables
- File upload validation
- Rate limiting for sensitive endpoints where appropriate
- OTP expiration
- Login error handling
- No password in API responses
- No secrets committed to Git

Never put:

JWT secret

AI API key

SMTP password

Database password

Judge0 API key

inside frontend code.

62. Configuration

Use environment variables for secrets.

Example:

DB_URL

DB_USERNAME

DB_PASSWORD

JWT_SECRET

JWT_EXPIRATION

AI_API_KEY

SMTP_HOST

SMTP_PORT

SMTP_USERNAME

SMTP_PASSWORD

JUDGE0_URL

JUDGE0_API_KEY

Provide:

.env.example

with placeholders.

Never provide actual secrets.

63. API Documentation

Use OpenAPI/Swagger.

Document:

- Authentication
- Student APIs

- Faculty APIs
- Admin APIs
- Attendance
- Marks
- Analytics
- AI
- Coding
- Placement
- Resume
- Interview
- Notifications

Swagger should allow authenticated API testing using JWT.

64. Testing

Backend:

- JUnit
- Mockito
- Spring Boot Test
- Repository tests
- Service tests
- Controller/API tests

Test:

Registration

Login

Duplicate email

Invalid password

JWT validation

Role authorization

Attendance

Marks

Eligibility

Placement application

Coding submission

Notifications

Frontend:

- Component tests where appropriate
 - API integration testing
 - Authentication flow testing
 - Form validation testing
-

65. Seed Data

Provide development seed data.

Create sample:

Admin

Faculty

Students

Departments

Courses

Subjects

Attendance

Marks

Companies

Jobs

Coding Problems

Contests

Announcements

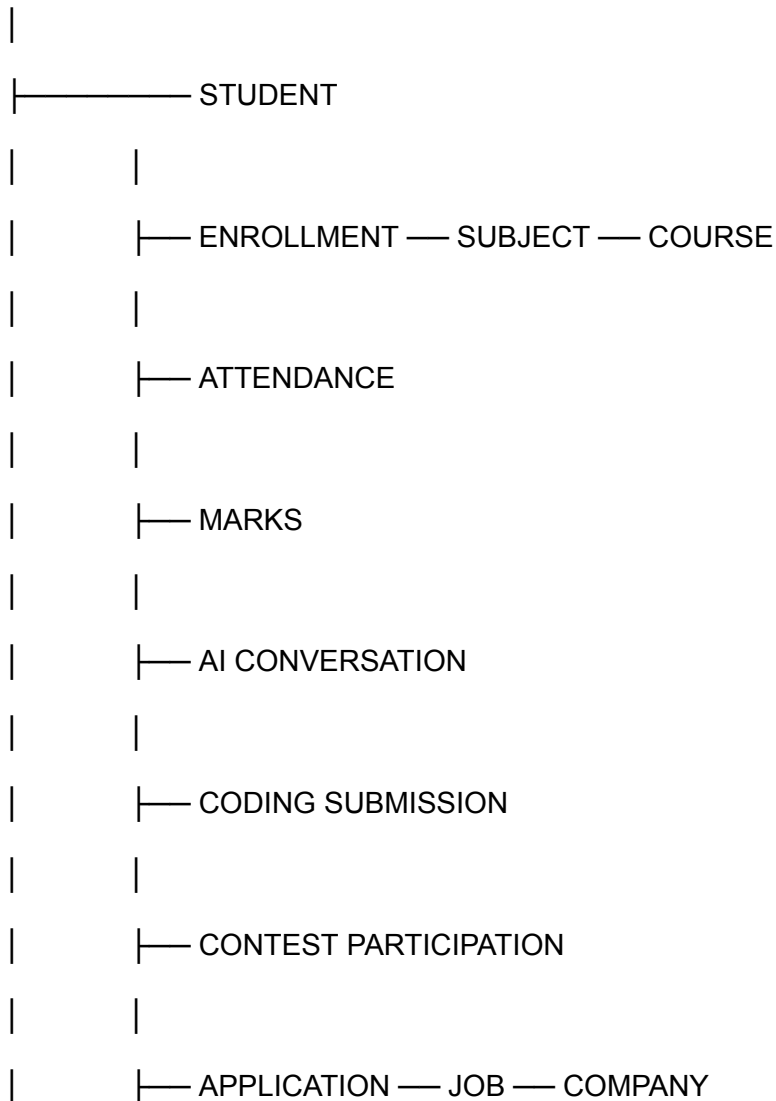
Passwords must be development-only and clearly documented.

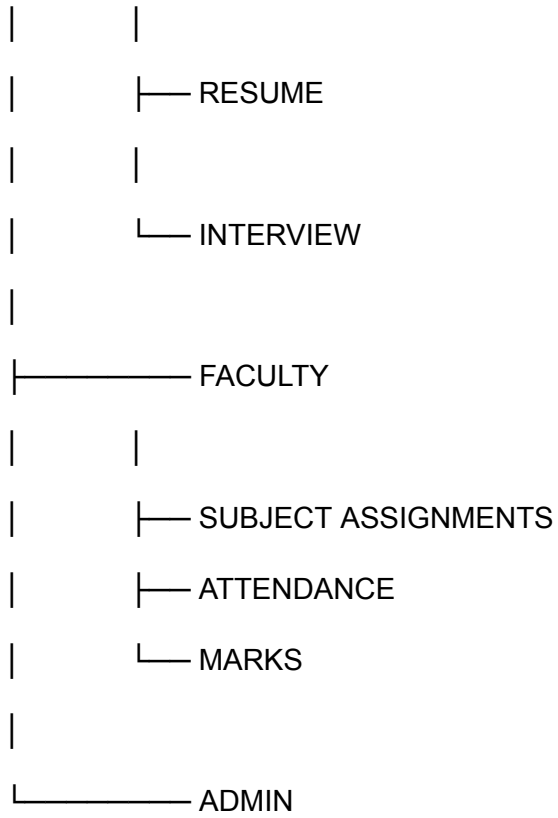
Do not use fake data in a way that makes the production system depend on it.

66. Database Relationships

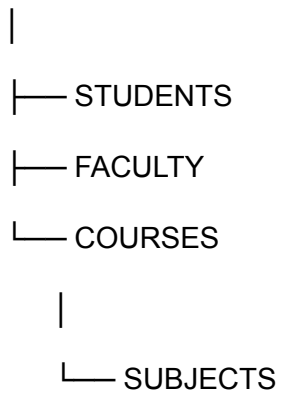
The overall relationship model should approximately be:

USER

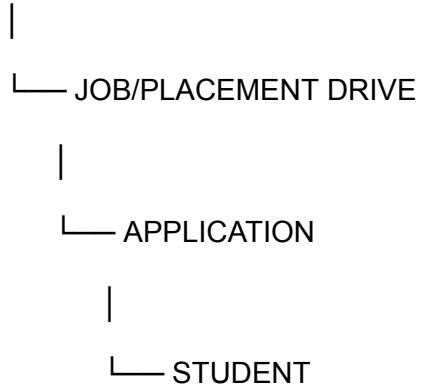




DEPARTMENT



COMPANY



67. Required Database Entities

At minimum, implement:

User

Student

Faculty

Admin

Department

Course

Subject

Enrollment

Attendance

Marks

Exam

Notification

Announcement

AIConversation

AIMessage

CodingProblem

CodingSubmission

CodingContest

ContestProblem

ContestParticipant

Company

Job

PlacementApplication

Resume

Education

ResumeProject

Experience

Certification

Skill

Achievement

Interview

OTP / PasswordResetToken

If **Admin** does not require separate profile data, it may be represented by the User role rather than a separate physical table. Do not create redundant tables without a functional reason.

68. Important Architecture Rule

Do not build this as a collection of disconnected features.

Every module must integrate with the central user and academic system.

For example:

Student



Academic Data



Analytics



AI Recommendations



Career Preparation



Placement

The student's data should flow through the system.

69. No Fake Functionality

The generated project must NOT contain:

- Fake login

- Hard-coded dashboard numbers
- Fake AI responses
- Fake coding execution
- Fake placement applications
- Fake notifications
- Static leaderboard
- Mock-only API responses
- Buttons that do nothing
- Frontend-only authentication

Every visible feature should either be fully functional or clearly marked as intentionally unavailable because an external service credential is required.

70. External Services

The system may depend on:

AI

Gemini API or OpenAI API.

Backend-only integration.

Code Execution

Judge0.

Backend-only integration.

Email

SMTP provider for:

- OTP
- Password reset
- Important notifications

Database

MySQL.

External services must be abstracted behind backend service classes so providers can be changed later.

For example:

AIService

└─ GeminiAIService

CodeExecutionService

└─ Judge0Service

EmailService

└─ SMTPEmailService

71. Deployment Architecture

Final deployment should support:

Internet

↓

React Frontend

↓

Spring Boot Backend

↓

MySQL

External:

Spring Boot

└─ AI API

└─ Judge0

|— SMTP

|— WebSocket

Use environment variables for deployment configuration.

72. Git/GitHub Structure

Repository:

SmartCampusERP/

|— backend/

|— frontend/

|— README.md

|— .gitignore

|— .env.example

Never commit:

.env

API keys

Passwords

JWT secrets

Database credentials

target/

node_modules/

73. README Requirements

Create a comprehensive README containing:

Project overview

Features

Architecture

Tech stack

Prerequisites

Installation

Database setup

Backend setup

Frontend setup

Environment variables

API documentation

Authentication

Running the application

Testing

Deployment

Folder structure

Screenshots

Future enhancements

74. Development Order

Build the project in this exact order to minimize integration problems:

Phase 1 — Foundation

Project setup

Backend

Frontend

MySQL

Git

Environment configuration

Phase 2 — Authentication :

User

Role

Registration

BCrypt

Login

JWT

JWT Filter

Authorization

OTP password reset

Phase 3 — Core Academic ERP :

Department

Course

Subject

Student

Faculty

Enrollment

Phase 4 — Academic Operations :

Attendance

Marks

Exams

Academic dashboard

Phase 5 — Analytics :

Performance calculations

Attendance analytics

GPA/CGPA

Risk detection

Chart.js dashboards

Phase 6 — AI :

AI service

AI assistant

Conversation history

Study plans

Practice questions

Phase 7 — Coding :

Coding problems

Code editor

Judge0

Submissions

Coding contests

Leaderboard

Phase 8 — Placement :

Companies

Jobs

Eligibility

Applications

Shortlisting

Placement dashboard

Phase 9 — Resume :

Resume

Education

Skills

Projects

Experience

Certifications

PDF generation

Phase 10 — Interview :

Question bank

AI interview preparation

Interview scheduling

Interview notifications

Phase 11 — Real-Time :

WebSocket

Notifications

Announcements

Live leaderboard

Phase 12 — Finalization :

Testing

Security

Validation

Error handling

Performance

Responsive UI

Documentation

Deployment

75. Definition of Done

The project is considered complete only when:

Authentication

- Registration works
- Login works
- Passwords are hashed
- JWT is generated
- JWT is validated
- Protected APIs reject unauthenticated users
- Roles work correctly
- OTP password reset works

Student

- Student can log in
- Student can view profile
- Student can view attendance
- Student can view marks
- Student can view analytics
- Student can use AI assistant
- Student can code
- Student can participate in contests
- Student can view leaderboard
- Student can view placement opportunities
- Student can apply
- Student can build resume
- Student can prepare for interviews

- Student can view interview schedules
- Student receives notifications

Faculty

- Faculty login works
- Faculty can manage authorized students
- Faculty can manage subjects
- Faculty can record attendance
- Faculty can enter marks
- Faculty can view performance analytics
- Faculty can send authorized announcements

Admin

- Admin login works
- Admin manages users
- Admin manages departments
- Admin manages courses
- Admin manages subjects
- Admin manages placement drives
- Admin manages companies
- Admin manages contests
- Admin manages announcements
- Admin views system analytics

Integration

All modules must communicate through real APIs and real database persistence.

76. Final Requirement to the AI Project Builder

Do not generate only a frontend mockup.

Generate a complete full-stack application with:

React frontend



REST APIs



Spring Boot services



JPA/Hibernate



MySQL

with:

JWT Security

Role-based authorization

Validation

Exception handling

DTOs

Database relationships

AI integration

Judge0 integration

Email/OTP

WebSockets

Analytics

File handling

Testing

Documentation

Every module must be integrated with the authentication system and database.

The application should be modular, maintainable, scalable and suitable for a real college ERP.

Do not skip backend implementation for any frontend feature. Do not use hard-coded data where database data is expected. Do not expose secrets in frontend code. Do not return passwords. Do not allow users to access resources belonging to other users unless their role explicitly permits it.

The final application should be runnable locally using documented commands and should include all required database setup, environment configuration, API documentation, seed data and testing instructions.

The core idea in one architecture

